centriMembSolver
User Guide

Vítor Geraldes
Department of Chemical Engineering
Instituto Superior Técnico, University of Lisbon
vitor.geraldes@tecnico.ulisboa.pt

Version 1.1.2
May 10, 2026

A finite-volume solver for incompressible flow with solute and tracer transport across rotating membranes, derived from `buoyantBoussinesqPimpleFoam` (OpenFOAM® v2506).

License and trademarks

Copyright. © 2026 Vítor Geraldes.

Software. `centriMembSolver` is distributed under the GNU General Public License version 3 (GPLv3) or later; see the `LICENSE` file in the source distribution. The software is provided "as is", WITHOUT WARRANTY OF ANY KIND, express or implied.

Documentation. This guide is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

Trademarks. OpenFOAM® is a registered trademark of OpenCFD Limited (ESI Group). Linux® is a registered trademark of Linus Torvalds. ParaView is a trademark of Kitware Inc. This work is not affiliated with, endorsed by, or sponsored by OpenCFD Limited, the Linux Foundation, or Kitware Inc.

Acknowledgements. `centriMembSolver` is derived from `buoyantBoussinesqPimpleFoam` of the OpenFOAM® distribution (OpenCFD Ltd., ESI Group; © 2011–2017 OpenFOAM Foundation, © 2021 OpenCFD Ltd.). The author thanks the collaborators whose discussions shaped the solver's scope.

How to cite

`centriMembSolver` is distributed as an open-source platform for the community of rotating-membrane simulation. Users who publish work based on the solver are encouraged to cite the versioned software release directly:

> Geraldes, V., *centriMembSolver: a finite-volume solver for rotating membrane systems built on OpenFOAM® technology*, version 1.1.2, Zenodo, 2026, doi:10.5281/zenodo.20109156 (concept DOI, resolves to the latest version; v1.1.2 archive: doi:10.5281/zenodo.20109390).

A BibTeX entry is provided in the source repository (`github.com/vitormgeraldes/centriMembSolver`). Pull requests with new boundary conditions, transport models, or tutorials are welcome.

Contents

Notation **5**

1 Introduction **7**

1.1 Motivation and scope . . . 7

1.2 Guide organization . . . 7

1.3 Changelog . . . 7

2 Installation **8**

2.1 Compatibility and requirements . . . 8

2.2 Build . . . 8

3 Theory and governing equations **9**

3.1 Incompressible Navier–Stokes in a rotating frame . . . 9

3.2 Boussinesq approximation . . . 9

3.3 The `gh`/`p_rgh` construction . . . 9

3.4 Working momentum equation: derivation . . . 10

3.5 Solute and passive-tracer transport . . . 11

3.6 Transport models in this solver . . . 11

4 Solver overview **13**

4.1 Time loop . . . 13

4.2 Momentum equation . . . 14

4.3 Pressure correction . . . 14

4.4 Field structure . . . 15

4.5 Diagnostics . . . 15

5 Boundary conditions **16**

5.1 Membrane physics . . . 16

5.2 Solvent flux boundary (`U`) . . . 17

5.3 Solute flux boundary (`CA`) . . . 18

5.4 Tracer flux boundary (`CT`) . . . 19

5.5 Patch dictionary summary . . . 20

6 Tutorial: `rotatingSlit` **22**

6.1 Geometry and mesh . . . 22

6.2 Transport properties . . . 22

6.3 Boundary conditions . . . 22

6.4 Running the case . . . 22

6.5 Real outputs . . . 22

7 Practical guidance **24**

8 References **25**

Notation

The mathematical symbols used in this guide map to OpenFOAM field names and dictionary keys as follows.

Fields and frame.

Symbol	Quantity	Field / dict key	Units
$\mathbf{U}$	velocity field (rotating frame)	`U`	$\mathrm{m\,s^{-1}}$
p_{rgh}	modified (dynamic) pressure	`p_rgh`	Pa
p	physical pressure (reconstructed)	`p`	Pa
gh	combined gravitational + centrifugal hydrostatic potential per unit mass	`gh`	$\mathrm{m^2\,s^{-2}}$
ρ_k	dimensionless density ratio ρ/ρ_0	`rhok`	–
$\mathbf{r}$	position vector relative to the rotation axis	—	m
$\mathbf{r}_\perp$	component of $\mathbf{r}$ perpendicular to the axis	—	m
$\boldsymbol{\Omega}$	angular-velocity vector along rotation axis	`SRFProperties`	$\mathrm{rad\,s^{-1}}$
Ω	angular speed, $\Omega = \lvert\boldsymbol{\Omega}\rvert$	—	$\mathrm{rad\,s^{-1}}$
r_m	membrane orbit radius	—	m
a_m	centrifugal acceleration at the membrane, $a_m = \Omega^2 r_m$	—	$\mathrm{m\,s^{-2}}$

Solute and tracer.

C_A	primary solute concentration	`CA`	$\mathrm{kg\,m^{-3}}$
$D_A(C_A)$	primary solute diffusivity (concentration–dependent)	`speciesData` (`DAB0..4`)	$\mathrm{m^2\,s^{-1}}$
$\nu(C_A)$	kinematic viscosity (concentration–dependent)	`transportProperties` (`nu0..2`)	$\mathrm{m^2\,s^{-1}}$
β	solutal contraction coefficient $(1/\rho_0)(\partial\rho/\partial C_A)$	`transportProperties` (`beta`)	$\mathrm{m^3\,kg^{-1}}$
ρ_0	reference density of the solvent at T_{ref}	`transportProperties` (`rho0`)	$\mathrm{kg\,m^{-3}}$
$\pi(C_A)$	osmotic pressure	via `virialCoeffs` on the BC	Pa
C_T	tracer (passive secondary species) concentration	`CT`	$\mathrm{kg\,m^{-3}}$
D_T	tracer diffusivity (constant)	`transportProperties` (`DT0`)	$\mathrm{m^2\,s^{-1}}$

Membrane.

J_v	solvent (water) flux through membrane	patch field `Jv`	$\mathrm{m\,s^{-1}}$
J_s	solute mass flux through membrane	patch field `Js`	$\mathrm{kg\,(m^2\,s)^{-1}}$
C_w, C_p	solute concentrations at the wall (feed side) and permeate side	internal / `CAp`	$\mathrm{kg\,m^{-3}}$
C_b	bulk reference concentration for diagnostics	dict (`CAb`)	$\mathrm{kg\,m^{-3}}$
A_h	hydraulic permeability of membrane	`Ah`	$\mathrm{m\,(Pa\,s)^{-1}}$
σ	solute reflection coefficient (Spiegler–Kedem)	`sigma`	–
B	solute permeability coefficient (Kedem–Katchalsky)	`B`	$\mathrm{m\,s^{-1}}$
R_{int}, R_{obs}	intrinsic and observed solute rejection	`Rint`, `Robs`	–
Γ	concentration polarisation factor $(C_w - C_b)/C_b$	`Gama`	–

1 Introduction

1.1 Motivation and scope

`centriMembSolver` is a finite-volume solver for transient, incompressible flow with solute and passive-tracer transport across selective membranes in a uniformly rotating frame. It is derived from `buoyantBoussinesqPimpleFoam` of the OpenFOAM® v2506 distribution and adds:

- explicit treatment of Coriolis and centrifugal forces in the momentum equation, with the centrifugal hydrostatic component absorbed into the modified pressure `p_rgh` (§3.3);
- a custom `CAViscosity` kinematic-viscosity model with a polynomial dependence on solute concentration;
- three custom membrane boundary conditions (`membraneSolventFlux`, `membraneSoluteFlux`, `membraneTracerFlux`), each offering three Spiegler–Kedem / Kedem–Katchalsky modeling choices;
- built-in conservation diagnostics and per-patch flux logging.

The solver targets direct numerical simulation (DNS) of rotating membrane systems where the coupling between centrifugal buoyancy, solute transport, and selective wall fluxes determines local mass transfer. Typical applications include centrifugal nanofiltration and reverse osmosis (NF/RO) cartridges, ultra-scale-down membrane testing rigs, and, more generally, any incompressible-flow setup in which a Boussinesq buoyancy approximation under rotation is acceptable.

1.2 Guide organization

- §2 – compilation and dependencies.
- §3 – governing equations, the `gh`/`p_rgh` construction, and transport models.
- §4 – overall solver structure: time loop, field declarations, and headers.
- §5 – the three membrane boundary conditions, the Robin closure they implement, and the dictionary syntax of every modeling choice.
- §6 – a worked example (`rotatingSlit`) that exercises the full pipeline and whose real outputs are reproduced here.
- §7 – practical guidance on stability and parallel execution.

1.3 Changelog

Version	Notes
1.1.2 (2026-05)	Cleanup release: legacy `doc/archive/` removed from the source tree. No code changes.
1.1.1 (2026-05)	User guide rewritten from scratch (rheoTool-style, code-first); sign-error fixes in BC code comments; `.zenodo.json` and `CITATION.cff` added.
1.1.0 (2026-05)	Local Time Stepping (LTS) for steady-state runs; hardened membrane BC classes (fail-fast on unknown selections, DAB boundary update fix, English comments); GPLv3 `LICENSE` added.
1.0.0 (2026-01)	Initial public release.

2 Installation

2.1 Compatibility and requirements

`centriMembSolver` compiles against OpenFOAM® v2506 or later (ESI/OpenCFD distribution). An active OpenFOAM environment is assumed throughout this guide – typically obtained by sourcing the corresponding `etc/bashrc`. GCC 9 or later and OpenMPI 4 (or any OpenFOAM-supported MPI implementation) are sufficient on Linux; the solver has been developed and tested on Linux distributions only. ParaView (with the `paraFoam` reader) is recommended for post-processing.

2.2 Build

The source tree contains two subprojects:

- `src/` – libraries: membrane boundary-condition classes (`libmembraneBCs.so`) and the concentration-dependent viscosity model (`libCATransportModels.so`);
- `solver/` – the application `centriMembSolver`.

Compile both, in this order:

```
cd <repo>/src     && wmake all libso
cd <repo>/solver  && wmake
```

A successful build places `centriMembSolver` in `$FOAM_USER_APPBIN` and the libraries in `$FOAM_USER_LIBBIN`. Verify with `which centriMembSolver`.

3 Theory and governing equations

3.1 Incompressible Navier–Stokes in a rotating frame

In a frame rotating with constant angular velocity $\boldsymbol{\Omega}$ about an axis through the origin, the incompressible Navier–Stokes equation reads

$$\rho\left(\frac{\partial \mathbf{U}}{\partial t} + \mathbf{U}\cdot\nabla\mathbf{U}\right) = -\nabla p + \nabla\cdot\big(\mu\,\nabla\mathbf{U}\big) + \rho\,\mathbf{f}_{\mathrm{rot}}, \qquad \nabla\cdot\mathbf{U} = 0, \tag{1}$$

where $\mathbf{U}$ is the velocity in the rotating frame, p is the pressure, μ the dynamic viscosity, and the body force is the sum of the two pseudo-forces appearing in a uniformly rotating frame,

$$\mathbf{f}_{\mathrm{rot}} = -\,\boldsymbol{\Omega}\times(\boldsymbol{\Omega}\times\mathbf{r}) - 2\,\boldsymbol{\Omega}\times\mathbf{U}, \tag{2}$$

with $\mathbf{r}$ the position vector relative to the rotation axis. The first term is conservative and points radially outward, $-\boldsymbol{\Omega}\times(\boldsymbol{\Omega}\times\mathbf{r}) = +\Omega^2\mathbf{r}_\perp$; the second is non-conservative.

3.2 Boussinesq approximation

Under the Boussinesq approximation, density variations $|\rho_k - 1| \ll 1$ (with $\rho_k = \rho/\rho_0$ and ρ_0 a constant reference density) are retained *only* in the body force. Inertia, viscous, and Coriolis terms use the constant ρ_0. The flow is treated as divergence-free.

The solver assumes a linear equation of state $\rho_k = 1 + \beta\,C_A$, where β is a constant solutal contraction coefficient $\beta = (1/\rho_0)(\partial\rho/\partial C_A)$. In the implementation, the constant offset (corresponding to the bulk contribution to ρ_k) is absorbed into the modified pressure `p_rgh` (see §3.3); only $\nabla\rho_k = \beta\,\nabla C_A$ is dynamically relevant, so the choice of reference state for ρ_k does not affect the velocity field.

3.3 The `gh`/`p_rgh` construction

Both gravity and the centrifugal acceleration of the rotating frame are conservative body forces and can therefore be absorbed into a single hydrostatic potential per unit mass, while their density-fluctuation counterparts are retained as the buoyancy drivers. This construction is classical for rotating Boussinesq flows (Chandrasekhar 1961 [3]; Lopez *et al.* 2013 [4]); the centrifugal term admits the gradient form

$$-\,\rho_0\,\boldsymbol{\Omega}\times(\boldsymbol{\Omega}\times\mathbf{r}) = \nabla\Big(\tfrac{1}{2}\rho_0\,|\boldsymbol{\Omega}\times\mathbf{r}|^2\Big), \tag{3}$$

so that the leading-order centrifugal load is moved into the pressure gradient and only the density-fluctuation part $-\rho'\,\boldsymbol{\Omega}\times(\boldsymbol{\Omega}\times\mathbf{r})$ remains as a buoyancy source.

`centriMembSolver` stores the combined gravitational and centrifugal hydrostatic potential in OpenFOAM's existing `gh` field, and the associated modified pressure in `p_rgh`. The explicit construction (from `solver/readRotationalAcceleration.H`) is

```
gh = (g & mesh.C())
   + 0.5*magSqr( omega ^ (mesh.C() - origin) )
   - ghRef;
```

with $\boldsymbol{\Omega}$ derived from `rpm` and `axis` in `SRFProperties`, and $\mathbf{x}_0$ from the `origin` field of the same dictionary. The reference offset $\mathrm{ghRef} = -|\mathbf{g}|\,\mathrm{hRef} + \frac{1}{2}\,\Omega^2\,r_{\mathrm{Ref}}^2$ sets the level surface; gradients of `gh` are unaffected by this constant.

The reconstructed physical pressure is $p = \rho_0(p_{\mathrm{rgh}} + \rho_k\,\mathrm{gh}) + p_{\mathrm{ref}}$, computed per write step in `pReconstruct.H` for diagnostics and downstream post-processing.

3.4 Working momentum equation: derivation

We collect the pieces of §§3.1–3.3 into the working form solved by the application.

Boussinesq raw form. Starting from (1) and (2), applying the Boussinesq approximation ($\rho \to \rho_0$ in inertia, viscous and Coriolis terms; $\rho = \rho_0\rho_k$ retained only in the body force) and using $\mathbf{a}_{\text{eff}} \equiv \mathbf{g} - \boldsymbol{\Omega}\times(\boldsymbol{\Omega}\times\mathbf{r}) = \nabla(\text{gh})$ for the conservative part of the body force gives

$$\rho_0\left(\frac{\partial\mathbf{U}}{\partial t} + \mathbf{U}\cdot\nabla\mathbf{U} + 2\,\boldsymbol{\Omega}\times\mathbf{U}\right) = -\nabla p + \nabla\cdot(\mu\,\nabla\mathbf{U}) + \rho_0\rho_k\,\mathbf{a}_{\text{eff}}. \tag{4}$$

Here only the buoyancy term carries the density factor ρ_k as required by the Boussinesq approximation; the inertia, viscous and Coriolis terms use the constant reference density ρ_0.

Modified pressure. The hydrostatic background of (4) is large in rotating membrane systems ($\Delta p_{\text{hyd}} = \frac{1}{2}\rho_0\Omega^2 r_m^2$ can reach ∼60 bar), while the dynamic perturbation of p is many orders of magnitude smaller. Solving for p directly therefore ill-conditions the pressure-correction step. Following the classical construction of §3.3 we split the physical pressure into a hydrostatic part and a dynamic part,

$$p \;=\; \rho_0\big(p_{\text{rgh}} + \rho_k\,\text{gh}\big) + p_{\text{ref}}, \tag{5}$$

where p_{rgh} is the unknown actually solved for in `pEqn.H`, and p_{ref} a constant. The pressure gradient unfolds as

$$\begin{aligned} -\nabla p &= -\rho_0\nabla p_{\text{rgh}} - \rho_0\,\nabla\big(\rho_k\,\text{gh}\big) \\ &= -\rho_0\nabla p_{\text{rgh}} - \rho_0\,\text{gh}\,\nabla\rho_k - \rho_0\rho_k\,\nabla(\text{gh}) \\ &= -\rho_0\nabla p_{\text{rgh}} - \rho_0\,\text{gh}\,\nabla\rho_k - \rho_0\rho_k\,\mathbf{a}_{\text{eff}}, \end{aligned} \tag{6}$$

where the second line uses the product rule and the third uses $\nabla(\text{gh}) = \mathbf{a}_{\text{eff}}$.

Cancellation and final form. Substituting (6) into the right-hand side of (4) cancels the two $\pm\rho_0\rho_k\,\mathbf{a}_{\text{eff}}$ terms and leaves

$$\rho_0\left(\frac{\partial\mathbf{U}}{\partial t} + \mathbf{U}\cdot\nabla\mathbf{U} + 2\,\boldsymbol{\Omega}\times\mathbf{U}\right) = -\rho_0\nabla p_{\text{rgh}} - \rho_0\,\text{gh}\,\nabla\rho_k + \nabla\cdot(\mu\,\nabla\mathbf{U}). \tag{7}$$

The hydrostatic load (gravity *and* centrifugal) has now left the unknown pressure: the only remaining centrifugal-force trace in the momentum balance is the buoyancy drive $-\rho_0\,\text{gh}\,\nabla\rho_k$, which is *non-conservative* and survives the gradient split.

Final form (per unit mass). Dividing (7) by the constant ρ_0 and introducing the kinematic viscosity $\nu = \mu/\rho_0$,

$$\frac{\partial\mathbf{U}}{\partial t} + \mathbf{U}\cdot\nabla\mathbf{U} + 2\,\boldsymbol{\Omega}\times\mathbf{U} = -\nabla p_{\text{rgh}} - \text{gh}\,\nabla\rho_k + \nabla\cdot\big(\nu\,\nabla\mathbf{U}\big). \tag{8}$$

This is the working momentum equation assembled and solved by the application in `UEqn.H` and `pEqn.H`. Three remarks on the structure of (8):

- The pressure unknown p_{rgh} contains only the *dynamic* pressure perturbation; the linear systems solved by `pEqn.H` are therefore well conditioned even at high rotation rates where the absolute pressure ranges over tens of bar.

- The buoyancy drive is the single non-conservative term $-\mathrm{gh}\,\nabla\rho_k$; it is discretised face-wise as $-\mathrm{gh}_f\,\mathrm{snGrad}(\rho_k)$, with gh_f the face interpolation of gh and `snGrad` the surface-normal gradient. In code this is the contribution `- ghf * fvc::snGrad(rhok)` that appears both in the momentum predictor and in the buoyancy flux of the pressure equation (`phig` in `pEqn.H`).
- The Coriolis term $-2\,\boldsymbol{\Omega}\times\mathbf{U}$ is non-conservative and cannot be absorbed into the modified pressure; it is added as an explicit vector source `coriolisSrc = -2.0*(omega ^ U)` refreshed at every PIMPLE outer iteration with the latest $\mathbf{U}$.

The kinematic viscosity ν is concentration-dependent in general and is retained *inside* the divergence operator; §3.6 describes its parameterisation.

3.5 Solute and passive-tracer transport

The solver always solves a transport equation for the primary solute C_A:

$$\frac{\partial C_A}{\partial t} + \nabla\cdot(C_A\mathbf{U}) = \nabla\cdot\big(D_{A,\mathrm{eff}}\,\nabla C_A\big), \qquad D_{A,\mathrm{eff}} = D_{AB}(C_A) + \nu_t/\mathrm{Sc}_t, \tag{9}$$

with ν_t the turbulent kinematic viscosity (zero for laminar runs) and Sc_t the turbulent Schmidt number (default 0.7, set in `transportProperties`). The molecular diffusivity $D_{AB}(C_A)$ is the polynomial described in §3.6; it is recomputed at every PIMPLE outer iteration on internal cells *and* on every patch face.

If `solveTracer true` is set under the `centriMembSolver` sub-dictionary of `controlDict` (or at the top level), an additional transport equation is solved for a passive tracer C_T, with constant molecular diffusivity D_T read from `transportProperties` (key `DT0`):

$$\frac{\partial C_T}{\partial t} + \nabla\cdot(C_T\mathbf{U}) = \nabla\cdot\big(D_{T,\mathrm{eff}}\,\nabla C_T\big), \qquad D_{T,\mathrm{eff}} = D_T + \nu_t/\mathrm{Sc}_t. \tag{10}$$

The tracer carries no contribution to the buoyancy and is integrated *after* the PIMPLE corrector loop, so its solution does not feed back into the velocity field.

3.6 Transport models in this solver

Concentration-dependent kinematic viscosity. The `CAViscosity` model implements a polynomial expansion

$$\nu(C_A) = \nu_0 + \nu_1 C_A + \nu_2 C_A^2 + \nu_3 C_A^3 + \nu_4 C_A^4, \tag{11}$$

with optional clamping to a user-specified $[\nu_{\min}, \nu_{\max}]$ range. The relevant subdictionary block in `constant/transportProperties` is

```
transportModel  CAViscosity;

CAViscosityCoeffs
{
    fieldName  CA;
    nu0        [0 2 -1 0 0 0 0]  1e-06;   // [m^2/s]
    nu1        [-1 5 -1 0 0 0 0] 1.61e-9; // [m^2/s] / [kg/m^3]
    nu2        [-2 8 -1 0 0 0 0] 0;
    clamp
    {
```

```
        min    [0 2 -1 0 0 0 0]  1e-08;
        max    [0 2 -1 0 0 0 0]  1e-03;
    }
}
```

The viscosity field is updated by `laminarTransport.correct()` inside `updateTransportProperties.H`.

Concentration-dependent diffusivity. A separate polynomial expansion gives

$$D_{AB}(C_A) = D_{AB,0} + D_{AB,1}C_A + D_{AB,2}C_A^2 + D_{AB,3}C_A^3 + D_{AB,4}C_A^4, \tag{12}$$

specified by the `diffusivity` subdictionary of `transportProperties`:

```
diffusivity
{
    DAB0   1.41e-9;
    DAB1  -2.661e-11;
    DAB2   1.3507e-12;
    DAB3  -1.8741e-14;
    DAB4   0;
}
```

The numerical evaluation clamps D_{AB} from below at 10^{-12} $\mathrm{m}^2\,\mathrm{s}^{-1}$ to avoid pathological negative values during the transient. The `DAB` field is recomputed both on internal cells and on every patch face at each PIMPLE outer step; the boundary update is critical on membrane patches where wall concentrations may differ substantially from the bulk.

Reference density and solutal contraction. The reference density ρ_0 and the solutal contraction coefficient β are read from the `density` subdictionary,

```
density
{
    rho0   998.23;     // [kg/m^3]
    beta   6.713e-4;   // [m^3/kg]   (NaCl example)
}
```

These feed the Boussinesq closure $\rho_k = 1 + \beta\, C_A$ used in the buoyancy term.

4 Solver overview

4.1 Time loop

The application source `solver/centriMembSolver.C` expresses the time loop as the following PIMPLE structure:

```
while (runTime.run())
{
    if (LTS)  { #include "setRDeltaT.H"  }
    else      { #include "CourantNo.H"
                #include "setDeltaT.H" }
    ++runTime;

    while (pimple.loop())                  // outer PIMPLE iteration
    {
        #include "UEqn.H"                   // assemble + solve momentum
        #include "CAEqn.H"                  // solute transport
        #include "updateTransportProperties.H"   // nu(C), DAB(C), rhok

        while (pimple.correct())            // inner pressure correctors
            #include "pEqn.H"

        if (pimple.turbCorr()) turbulence->correct();
    }

    #include "CTEqn.H"                      // tracer (post-PIMPLE)
    #include "pReconstruct.H"               // physical p
    #include "monitorConservation.H"        // global mass balance
    #include "Gama.H"                       // Gamma diagnostic
    #include "GamaT.H"                      // tracer Gamma diagnostic

    runTime.write();
}
```

A few features deserve mention:

- *LTS auto-detection.* At start-up, `createRDeltaT.H` (provided by OpenFOAM) inspects `fvSchemes` for any field using the `localEuler` time scheme; if found, the solver allocates the `rDeltaT` field and sets `LTS=true`, switching from CFL-based time-step control to local time stepping appropriate for steady-state pseudo-transient runs.
- *Solute coupling.* Both momentum and solute transport are solved at the start of each PIMPLE outer iteration, so that transport properties driven by C_A (viscosity, diffusivity, buoyancy) follow the velocity correction within the same iteration.
- *Tracer.* When `solveTracer=true`, the tracer equation is solved *once per time step*, after the PIMPLE loop converges. This reflects the passive nature of the tracer: it is advected by the converged velocity field and contributes neither to buoyancy nor to the momentum coupling.

4.2 Momentum equation

The momentum predictor (`UEqn.H`) is

```
coriolisSrc = -2.0*(omega ^ U);

fvVectorMatrix UEqn
(
    fvm::ddt(U)
  + fvm::div(phi, U)
  + turbulence->divDevReff(U)
  - coriolisSrc
 ==
    fvOptions(U)
);

UEqn.relax();
fvOptions.constrain(UEqn);

if (pimple.momentumPredictor())
{
    solve
    (
        UEqn ==
        fvc::reconstruct
        (
            ( -ghf*fvc::snGrad(rhok)
              -fvc::snGrad(p_rgh) )*mesh.magSf()
        )
    );
    fvOptions.correct(U);
}
```

The Coriolis source is built explicitly with the current $\mathbf{U}$ and refreshed at every outer PIMPLE iteration. The `momentumPredictor` flag in `fvSolution` should be disabled (`momentumPredictor no`) for DNS runs in strongly rotating flows, to avoid spurious Courant excursions induced by the predictor; CFL-bounded time steps then make a single outer corrector per time step normally sufficient.

4.3 Pressure correction

`pEqn.H` performs the pressure correction in standard buoyantBoussinesqPimple style. The buoyancy contribution to the flux is

```
surfaceScalarField phig(-rAUf*ghf*fvc::snGrad(rhok)*mesh.magSf());
```

which is added to the predicted face flux `phiHbyA` before the pressure Laplacian is solved. After the inner non-orthogonal correction loop, the velocity is updated by $\mathbf{U} \leftarrow H/A + \mathrm{rAU}\,\mathrm{reconstruct}((\phi_\mathrm{g} - \phi_\mathrm{p})/\mathrm{rAUf})$.

4.4 Field structure

`createFields.H` declares the runtime fields used by the solver. All fields are read from the case time directory unless otherwise stated.

Field	I/O	Role
`p_rgh`	MUST_READ / AUTO_WRITE	modified pressure (the unknown solved by `pE`
`U`	MUST_READ / AUTO_WRITE	velocity in rotating frame
`CA`	MUST_READ / AUTO_WRITE	primary solute concentration
`rhok`	READ_IF_PRESENT / AUTO_WRITE	dimensionless density ratio
`Deff`	not written	effective solute diffusivity field
`DAB`	not read / AUTO_WRITE	molecular diffusivity field, recomputed each it ation
`p`	not read / AUTO_WRITE	physical pressure (reconstructed)
`gh, ghf`	READ_IF_PRESENT / NO_WRITE	combined hydrostatic potential
`Js, Jv`	not read / AUTO_WRITE	solute and solvent fluxes through the membra
`CAp`	not read / AUTO_WRITE	permeate concentration
`Gama`	not read / AUTO_WRITE	polarisation factor $\Gamma = (C_w - C_b)/C_b$
`coriolisSrc`	not read / not written	Coriolis source $-2\,\mathbf{\Omega}\times\mathbf{U}$
Tracer fields, only allocated when `solveTracer=true`		
`CT`	MUST_READ / AUTO_WRITE	tracer concentration
`DT, DTeff`	not read / AUTO_WRITE	tracer diffusivity fields
`JsT, CTp, GamaT`	not read / AUTO_WRITE	tracer membrane diagnostics

4.5 Diagnostics

Conservation monitor. `monitorConservation.H` writes a global mass-balance summary at every solver write time to `postProcessing/conservation/cons.dat`, with columns t, M, $\mathrm{d}M/\mathrm{d}t$, m_{in}, m_{out}, m The relative residual relM provides a quick sanity check on conservation.

Polarisation diagnostic. `Gama.H` writes the local concentration polarisation factor $\Gamma = (C_w - C_b)/C_b$ into the volume field `Gama`, populated from the wall values of `CA` via the membrane BC's internal state. An analogous `GamaT` is computed for the tracer if `solveTracer=true`.

5 Boundary conditions

`centriMembSolver` ships three custom membrane boundary conditions, one per transported field:

- `membraneSolventFlux` for the velocity `U`;
- `membraneSoluteFlux` for the primary solute `CA`;
- `membraneTracerFlux` for the optional tracer `CT`.

Each class offers three modeling choices that specialise the underlying Spiegler–Kedem and Kedem–Katchalsky relations. The present section first recalls the membrane physics and then documents the dictionary syntax of every choice.

5.1 Membrane physics

Driving forces and osmotic pressure. Transmembrane transport is driven by the hydrostatic difference $\Delta p = p_f - p_p$ and the osmotic difference $\Delta\pi = \pi(C_f) - \pi(C_p)$. The osmotic pressure is expressed through a virial expansion in solute concentration,

$$\pi(C_A) = \sum_{k=1}^{N} a_k \, C_A^k, \tag{13}$$

with mass-based coefficients a_k in $\mathrm{Pa\,m^{3k}\,kg^{-k}}$ (C_A in $\mathrm{kg\,m^{-3}}$). At infinite dilution the linear term reduces to van 't Hoff, $a_1 = i\,RT/M$. The coefficients are specified per-patch via the `virialCoeffs` list of the solvent BC.

Spiegler–Kedem solvent flux. The solvent flux J_v ($\mathrm{m\,s^{-1}}$) follows

$$J_v = A_h \, (\Delta p - \sigma \, \Delta\pi), \tag{14}$$

with hydraulic permeability A_h (key `Ah`) and reflection coefficient σ (key `sigma`).

Kedem–Katchalsky solute flux. The solute mass flux J_s ($\mathrm{kg\,(m^2\,s)^{-1}}$) is

$$J_s = (1-\sigma)\, J_v \, C_w + B\,(C_w - C_p), \tag{15}$$

where B is the intrinsic solute permeability (key `B`), C_w the wall concentration on the feed side, and C_p the permeate concentration. The intrinsic rejection is, by definition, $R_{\mathrm{int}} \equiv 1 - C_p/C_w$.

Local mass balance. At the membrane face the convective influx, the diffusive removal back into the bulk feed, and the permeate outflux must balance:

$$J_v \, C_w \; - \; D_A \left.\frac{\partial C_A}{\partial n}\right|_w \; = \; J_s \; = \; J_v \, C_p, \tag{16}$$

where $\partial/\partial n$ is the outward normal derivative on the feed-side patch, evaluated discretely as $(C_w - C_i)/\delta_n$ (OpenFOAM's `snGrad`). Equation (16) couples C_w to the resolved interior field; no phenomenological mass-transfer coefficient is required, since the boundary layer is resolved by the DNS.

Closure 1 – intrinsic rejection. Specifying R_{int} gives $C_p = (1 - R_{\mathrm{int}})\, C_w$, which substituted in (16) yields

$$D_A \left.\frac{\partial C_A}{\partial n}\right|_w = R_{\mathrm{int}} \, J_v \, C_w. \tag{17}$$

Closure 2 – solute permeability. Combining (15) with $J_s = J_v\, C_p$ gives

$$C_p = \frac{B + (1-\sigma)\, J_v}{B + J_v}\, C_w, \quad \Longrightarrow \quad D_A \left.\frac{\partial C_A}{\partial n}\right|_w = \frac{\sigma\, J_v^2}{B + J_v}\, C_w. \tag{18}$$

Robin BC implementation. Both (17) and (18) reduce to the canonical Robin form $k\,(C_w - C_i) = h_{\text{phys}}\, C_w$ with $k = D_A/\delta_n$ and h_{phys} given by either $R_{\text{int}}\, J_v$ or $\sigma\, J_v^2/(B + J_v)$. Solving for C_w,

$$C_w = \frac{k}{k - h_{\text{phys}}}\, C_i. \tag{19}$$

In OpenFOAM this is realised through `mixedFvPatchScalarField` with reference value and reference gradient set to zero, and `valueFraction` $\alpha_{\text{vF}} = -\,h_{\text{phys}}/(k - h_{\text{phys}})$, which is *negative-valued* under physical operating conditions ($0 < h_{\text{phys}} < k$). The OpenFOAM identifier `valueFraction` therefore stores an algebraic Robin weight, not a fraction in $[0, 1]$.

5.2 Solvent flux boundary (`U`)

`membraneSolventFlux` prescribes the wall-normal component of **U** on the feed-side patch. Three modeling choices are available; in all of them the local solvent flux is clipped to non-negative values, $J_v \leftarrow \max(0, J_v)$, so that the membrane never admits permeate-to-feed back-flow.

(1) Spiegler–Kedem. Implements (14). Uses the user-supplied A_h and σ, and the osmotic pressures $\pi(C_f)$, $\pi(C_p)$ from the `virialCoeffs` of the patch.

```
membraneA
{
    type            membraneSolventFlux;
    model           SpieglerKedem;
    Ah              0.5e-11;        // [m/(Pa s)]
    sigma           1.0;            // [-]
    virialCoeffs    (2.5e3 0);      // a1 [Pa m^3/kg], a2 [Pa m^6/kg^2]
    pPermConst      0;              // [Pa]
    value           uniform (0 0 0);
}
```

(2) Constant flux. Prescribes a uniform J_v, independent of driving forces. Useful for benchmarking or when the membrane permeability is unknown.

```
membraneA
{
    type            membraneSolventFlux;
    model           constantFlux;
    Jv              3.0e-5;         // [m/s], outward-positive
    value           uniform (0 0 0);
}
```

(3) Target average flux. Tunes an effective hydraulic permeability A_h^* at every step so that the area-averaged J_v matches a user-prescribed `JvAverage`, while the local fluxes remain proportional to the local $\Delta p - \sigma\,\Delta\pi$. The calibrated A_h^* is constrained to a user range [`AhMin`, `AhMax`] and is logged at each write time to `postProcessing/membraneSolventFlux/<patch>/calibratedProperties.dat`.

```
membraneB
{
    type            membraneSolventFlux;
    model           targetAverageFlux;

    JvAverage       3.0e-5;         // target <Jv> [m/s]
    AhMin           1.0e-13;
    AhMax           1.0e-9;

    sigma           1.0;
    virialCoeffs    (7314.5 8.253);
    pPermConst      0;

    windowAverage   true;           // optional moving-average reporting
    Twindow         5;              // [s]

    value           uniform (0 0 0);
}
```

For all three models the per-write time-series of the area-averaged permeate flux is appended to `postProcessing/membraneSolventFlux/<patch>/permeateFlux.dat`.

5.3 Solute flux boundary (`CA`)

`membraneSoluteFlux` couples the solute transport to the solvent flux through the local mass balance of §5.1 and is implemented as a Robin (`mixedFvPatchScalarField`) BC.

(1) Intrinsic rejection. Specifies R_{int}; uses Eq. (17).

```
membraneB
{
    type     membraneSoluteFlux;
    model    intrinsicRejection;
    Rint     0.99;
    CAb      1.0;          // bulk reference [kg/m^3] for diagnostics
    value    uniform 0;
}
```

(2) Solute permeability. Specifies B (and optionally σ); uses Eq. (18). When `sigma` is omitted, the reflection coefficient is inherited from the solvent BC on `U`, defaulting to 1 if no membrane solvent BC is found.

```
membraneB
{
    type     membraneSoluteFlux;
```

```
    model    solutePermeability;
    B        1.0e-7;
    sigma    0.95;          // optional; inherited from U BC if omitted
    CAb      5.0;
    value    uniform 0;
}
```

(3) Observed rejection. Targets a user-prescribed observed rejection $R_{\text{obs}} = 1 - \langle J_v\, C_p \rangle / (\langle J_v \rangle\, C_b)$ and infers the corresponding R_{int} at each step via a safeguarded secant–bisection solver, then applies Eq. (17) face-by-face with the inferred R_{int}. Useful when matching experimental rejection data.

```
membraneB
{
    type      membraneSoluteFlux;
    model     observedRejection;
    Robs      0.80;             // target observed rejection
    RintMin   0.0;
    RintMax   0.9999;
    tol       1e-6;
    maxIter   50;
    CAb       5.0;
    value     uniform 0;
}
```

For all three models, per-face J_s, J_v and C_p are exposed through the patch fields `Js`, `Jv`, `CAp`, and the time series of R_{int} and R_{obs} is logged in `postProcessing/membraneSoluteFlux/<patch>/Rint_vs_time.dat` (columns: *time*, R_{int}, R_{obs}). The local polarisation factor is written to the volume field `Gama`.

5.4 Tracer flux boundary (`CT`)

`membraneTracerFlux` reuses the same Robin machinery applied to the passive tracer field C_T with diffusivity D_T and tracer-specific parameters $R_{T,\text{int}}$, B_T, σ_T. The BC is allocated only when `solveTracer=true`. All operations on C_T are independent of those on C_A; the only coupling is through the shared solvent flux J_v.

The three modeling choices (`intrinsicRejection`, `solutePermeability`, `observedRejection`) follow exactly the solute case of §5.3, with $C_A \to C_T$, $D_A \to D_T$, and tracer-specific dictionary keys (`RTint`, `BT`, `sigmaT`, `RTobs`, etc.). When `sigmaT` is omitted in the solute-permeability model, the default is $\sigma_T = 1$ (full rejection of the tracer); unlike the solute case, σ_T is *not* inherited from the solvent BC, since the reflection coefficient of a passive tracer is generally unrelated to that of the primary solute.

A typical tracer dictionary entry is

```
membraneB
{
    type      membraneTracerFlux;
    model     intrinsicRejection;
    RTint     1.0;              // perfectly rejected tracer
    CTb       1.0e-3;           // bulk reference [kg/m^3]
    value     uniform 0;
}
```

Diagnostic outputs mirror the solute case (`postProcessing/membraneTracerFlux/<patch>/RTint_vs_time.da` volume field `GamaT`).

5.5 Patch dictionary summary

Tables 1–3 list the principal dictionary entries.

Table 1: Solvent flux boundary entries (`membraneSolventFlux`).

Key	Units	Description / default
`model`	–	`SpieglerKedem` (default), `constantFlux`, `targetAverageFlux`
`Ah`	$\mathrm{m\,(Pa\,s)^{-1}}$	hydraulic permeability (SK)
`sigma`	–	reflection coefficient; default 1
`virialCoeffs`	$\mathrm{Pa\,m^{3k}\,kg^{-k}}$	$(a_1, a_2, \ldots)$ for $\pi(C_A)$
`Jv`	$\mathrm{m\,s^{-1}}$	constant flux magnitude (constantFlux)
`JvAverage`	$\mathrm{m\,s^{-1}}$	target area-averaged flux (targetAverageFlux)
`AhMin`, `AhMax`	$\mathrm{m\,(Pa\,s)^{-1}}$	bounds on calibrated A_h^*
`pPermConst`	Pa	permeate-side pressure reference; default 0
`windowAverage`	–	enable moving-average reporting; default `false`
`Twindow`	s	moving-average window length
`CAName`, `CApName`	–	feed and permeate concentration field names; default `CA`, `CAp`

Table 2: Solute flux boundary entries (`membraneSoluteFlux`).

Key	Units	Description / default
`model`	–	`intrinsicRejection` (default), `solutePermeability`, `observedRejection`
`Rint`	–	intrinsic rejection (intrinsicRejection)
`B`	$\mathrm{m\,s^{-1}}$	solute permeability (solutePermeability)
`sigma`	–	overrides solvent BC inheritance; default 1
`Robs`	–	target observed rejection (observedRejection)
`CAb`	$\mathrm{kg\,m^{-3}}$	bulk reference for Γ and R_{obs}
`RintMin`, `RintMax`	–	search bounds for inverse mode
`tol`, `maxIter`	–	inverse-search tolerance and max iterations
`UName`	–	velocity field name; default `U`

Table 3: Tracer flux boundary entries (`membraneTracerFlux`).

Key	Units	Description / default
`model`	–	`intrinsicRejection` (default), `solutePermeability`, `observedRejection`
`RTint`	–	intrinsic tracer rejection (intrinsicRejection)
`BT`	$\mathrm{m\,s^{-1}}$	tracer permeability (solutePermeability)
`sigmaT`	–	tracer reflection coefficient; default 1 (full rejection)
`RTobs`	–	target observed tracer rejection (observedRejection)
`CTb`	$\mathrm{kg\,m^{-3}}$	bulk reference for Γ_T and $R_{T,\mathrm{obs}}$
`RTintMin`, `RTintMax`	–	search bounds for inverse mode
`tol`, `maxIter`	–	inverse-search controls

6 Tutorial: `rotatingSlit`

The bundled `rotatingSlit` tutorial is a two-dimensional slit flow with two membrane segments, intended to exercise the full solver pipeline (mesh, BCs, transport models, diagnostics) on a case that runs in under one minute on a desktop.

6.1 Geometry and mesh

The geometry is a slit $L \times h = 20\,\mathrm{mm} \times 1\,\mathrm{mm}$ with the bottom wall split into two membrane patches at $x = 10\,\mathrm{mm}$. `system/blockMeshDict` discretises the slit into two structured hex blocks (left: 120×30, right: 80×30) with y-grading 20 toward the membrane. The patches are `membraneA` (left bottom), `membraneB` (right bottom), `top` (`symmetry`), `inlet`, `outlet`, and `frontAndBack` (`empty` for 2D).

6.2 Transport properties

`constant/transportProperties` configures `CAViscosity` with $\nu_0 = 10^{-6}\ \mathrm{m^2\,s^{-1}}$ (constant viscosity in this baseline tutorial), constant molecular diffusivity $D_{AB,0} = 10^{-9}\ \mathrm{m^2\,s^{-1}}$, $\rho_0 = 998.23\,\mathrm{kg\,m^{-3}}$, $\beta = 9.855{\times}10^{-4}\ \mathrm{m^3\,kg^{-1}}$, and turbulent Schmidt number $\mathrm{Sc}_t = 0.7$. `constant/SRFProperties` sets `rpm 0` (no rotation in this baseline run; the user is invited to set a non-zero value to exercise the rotating-frame extensions).

6.3 Boundary conditions

The active membrane is `membraneB` (the upstream segment `membraneA` is set to `noSlip` for the velocity and `zeroGradient` for C_A, leaving C_A unaffected by that segment). On `membraneB`:

- `U` uses `membraneSolventFlux` with `model targetAverageFlux`, `JvAverage` $= 3{\times}10^{-5}\,\mathrm{m\,s^{-1}}$, $\sigma = 1$, `virialCoeffs (7314.5 8.253)`, `AhMin` $= 10^{-13}$, `AhMax` $= 10^{-9}\ \mathrm{m\,(Pa\,s)^{-1}}$;
- `CA` uses `membraneSoluteFlux` with `model intrinsicRejection`, $R_{\mathrm{int}} = 0.99$, $C_b = 1.0\,\mathrm{kg\,m^{-3}}$.

At the inlet, $\mathbf{U}_{\mathrm{in}} = (0.01, 0, 0)\ \mathrm{m\,s^{-1}}$ and $C_{A,\mathrm{in}} = 5.0\ \mathrm{kg\,m^{-3}}$; `p_rgh` uses `fixedFluxPressure` on the membranes and `zeroGradient` elsewhere.

6.4 Running the case

```
cd <repo>/tutorials/rotatingSlit
blockMesh
centriMembSolver
```

The case writes every 1 s of simulated time and ends at $t = 30\,\mathrm{s}$. Diagnostics appear under `postProcessing/`: `conservation/cons.dat`, `membraneSolventFlux/membraneB/calibratedProperties.dat`, `membraneSoluteFlux/membraneB/Rint_vs_time.dat`, and the volume field `Gama`.

6.5 Real outputs

Figure 1 shows the actual time evolution observed when running the tutorial out of the box.

The two transients have similar time scales: the calibrated A_h^* converges (panel a) on the same $\sim$10–15 s window over which the polarised layer establishes (panel b). The gap between the prescribed $R_{\mathrm{int}} = 0.99$ and the steady $R_{\mathrm{obs}} \approx 0.60$ is the operational signature of concentration polarisation.

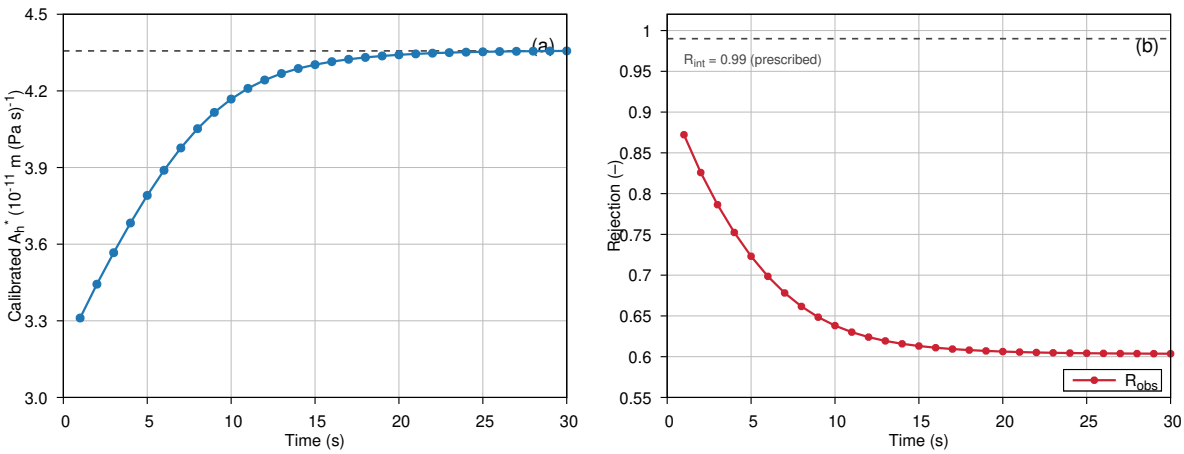

Figure 1: Real outputs of the bundled `rotatingSlit` tutorial (2D slit, $L \times h = 20\,\text{mm} \times 1\,\text{mm}$, $\sigma = 1$, $R_{\text{int}} = 0.99$, `JvAverage` $= 3\times10^{-5}\ \text{m}\,\text{s}^{-1}$). **(a)** The calibrated permeability $A_h^*(t)$ extracted from the patch dictionary at each write time; the `targetAverageFlux` model converges to the steady operating permeability $\sim 4.36\times10^{-11}\ \text{m}\,(\text{Pa}\,\text{s})^{-1}$ within $\sim$15 s. **(b)** Time evolution of the observed rejection R_{obs} (red), starting near the prescribed $R_{\text{int}} = 0.99$ (dashed) and stabilising around 0.60 as the polarised layer establishes.

7 Practical guidance

Stability and time stepping. With Coriolis treated explicitly, the convective Courant condition is the dominant time-step constraint; keep the Courant number below ~ 1 near the membrane. When using `targetAverageFlux`, choose `AhMin` and `AhMax` narrowly around the expected steady operating permeability: a range that is too wide allows the calibrated A_h^* to wander outside the physical regime and may produce oscillatory behaviour. When using `observedRejection`, pick a tight tolerance `tol` and verify that the target R_{obs} lies inside the physically realistic interval $[R_{\text{int,min}}, R_{\text{int,max}}]$ – otherwise the inverse search hits the bracket and reports a clamped value rather than the target. For DNS in strongly rotating flows, set `momentumPredictor no` in `system/fvSolution` to avoid spurious Courant excursions induced by the predictor.

Parallel execution. For three-dimensional cases the solver runs under MPI in the standard OpenFOAM idiom:

```
decomposePar
mpirun -np <N> centriMembSolver -parallel
reconstructPar
```

The number of processes is set by `numberOfSubdomains` in `system/decomposeParDict`; `simple` or `scotch` decomposition methods are supported. All membrane boundary conditions and diagnostics use parallel reductions (`Pstream::reduce`, `returnReduce`). For complex geometries or very fine meshes, however, it is advisable to invoke `redistributePar` after `decomposePar` so that each membrane patch is owned by a single processor; this avoids subtle load-balancing artefacts in the area-weighted averages. The conservation monitor (`postProcessing/conservation/cons.dat`) provides a quick sanity check on mass balances across processor boundaries.

8 References

[1] Spiegler, K. S. & Kedem, O. (1966). Thermodynamics of hyperfiltration (reverse osmosis): criteria for efficient membranes. *Desalination*, 1, 311–326.

[2] Wu, J. J. (2019). On the application of the Spiegler–Kedem model to forward osmosis. *BMC Chemical Engineering*, 1, article 15. doi:10.1186/s42480-019-0014-8.

[3] Chandrasekhar, S. (1961). *Hydrodynamic and Hydromagnetic Stability.* Clarendon Press, Oxford. International Series of Monographs on Physics. (Classical reference for absorbing the centrifugal term into the pressure gradient in rotating Boussinesq flows.)

[4] Lopez, J. M., Marques, F. & Avila, M. (2013). The Boussinesq approximation in rapidly rotating flows. *Journal of Fluid Mechanics*, 737, 56–77. doi:10.1017/jfm.2013.558. (Modern formulation of the gravity + centrifugal absorption used by this solver.)